# Empirica: Epistemic Self-Assessment for AI Systems

Measuring Knowledge State Through Functional Metacognition

David Samuel L. Van Assche
Empirica
david@getempirica.com

February 2, 2026

**Abstract**

We present Empirica, a framework for AI epistemic self-assessment that operationalizes the principle that AI systems must investigate before acting. Through 479,686 evidence observations across 638 sessions, we demonstrate that AI capability genuinely increases during task execution—the learning delta is real, not an artifact of assessment correction. Filtering to 267 clean sessions (excluding default assessments and incomplete workflows), 93.6% showed knowledge improvement with a mean capability growth of +0.200. We introduce a 13-dimensional epistemic vector representation capturing what AI systems know, what they can do, and how ready they are to act. Bayesian calibration analysis reveals unified confidence growth across all vectors during learning, with calibration variance dropping 115× as evidence accumulates. The framework implements epistemic gates that enforce investigation when knowledge is insufficient, preventing the "overconfidence trap" where AI systems act on incomplete knowledge. We further introduce dual-track calibration: self-referential corrections from learning deltas alongside grounded corrections from post-test evidence (test results, git metrics, goal completion), partially addressing the open problem of externally validating AI self-assessment. We release Empirica as open-source infrastructure for epistemic gating, calibration tracking, and structured AI workflows.

**Keywords:** AI self-assessment, epistemic vectors, metacognition, calibration, uncertainty quantification

## 1 Introduction

### 1.1 The Execution-Layer Fallacy

The dominant paradigm for deploying AI systems treats them as execution layers: a user provides a task, the AI executes it, and returns output. This model—borrowed from traditional software engineering—assumes capability is fixed and known. Give the system an input; receive a deterministic output.

This assumption fails catastrophically for large language models.

When an LLM receives a complex task, it does not possess fixed, queryable knowledge. Its effective capability on *that specific task* depends on factors that emerge only through engagement: what context it accumulates, what connections it discovers, what it learns by exploring the problem space. An LLM asked to "fix the authentication bug" may have latent capability to solve it—but that capability becomes *actual* only through investigation.

The execution-layer model ignores this. It assumes the AI either can or cannot perform the task, and proceeds accordingly. The result is a familiar failure mode:

- AI reports low uncertainty (it "thinks" it knows)
- AI has low actual capability (it doesn't actually know)

- AI proceeds without investigation
- AI produces confident, wrong output
- User loses trust

We call this the **overconfidence trap**: the AI's self-assessed certainty exceeds its actual capability, leading to premature action without adequate exploration.

### 1.2 Noetic Before Praxic

This paper proposes a different model: **AI systems must investigate before acting**. We formalize this as two phases of AI cognition:

**Noetic phase:** Exploration, hypothesis formation, knowledge acquisition. The AI investigates the problem, reads relevant context, identifies unknowns, and builds genuine understanding.

**Praxic phase:** Execution, implementation, action. The AI applies its acquired knowledge to produce output.

The critical insight is that these phases are not optional or interchangeable. *Noetic must precede praxic.* An AI that acts without first investigating is an AI that acts on incomplete knowledge—and incomplete knowledge produces errors.

This is not a novel observation to practitioners who work with AI systems daily. It is, however, surprisingly absent from the research literature, which tends to evaluate AI on fixed benchmarks rather than dynamic, exploratory tasks.

### 1.3 The Learning Delta

If noetic precedes praxic—if investigation precedes action—then a measurable phenomenon should emerge: **AI capability should increase during task execution**.

An AI that begins a task with partial knowledge, investigates, and builds understanding will assess its own epistemic state differently at task-end than at task-start. The difference between these assessments—the *learning delta*—measures genuine knowledge acquisition.

We present evidence from 479,686 observations across 638 sessions showing exactly this pattern. Across all 13 epistemic dimensions we track, AI systems show systematic confidence growth during task execution:

| **Phase** | **Mean Capability Assessment** |
|---|---:|
| Pre-task (PREFLIGHT) | 0.663 |
| Post-task (POSTFLIGHT) | 0.863 |
| **Learning Delta** | **+0.200** |

Table 1: Learning delta across 267 clean sessions.

Filtering to 267 clean sessions (excluding default assessments and incomplete workflows), **93.6% showed knowledge improvement**. This is not calibration error being corrected. This is learning being measured.

### 1.4 Empirica: Epistemic Infrastructure for AI

We introduce **Empirica**, a framework that operationalizes noetic-before-praxic through four mechanisms:

1. **Epistemic Vectors**: A 13-dimensional representation of AI self-assessed cognitive state, covering knowledge (KNOW), uncertainty (UNCERTAINTY), engagement (ENGAGEMENT), and 10 additional dimensions derived from computational epistemology.

2. **Epistemic Gates**: Decision points that evaluate whether the AI has sufficient knowledge to proceed. Gates enforce investigation when knowledge is low, preventing premature action.

3. **The CASCADE Workflow**: A 4-phase loop—PREFLIGHT → CHECK → POST-FLIGHT → POST-TEST—that captures epistemic state before and after task execution, measures learning, and calibrates future assessments through dual-track Bayesian updates.

4. **Grounded Verification**: Post-test evidence collection that compares self-assessed epistemic state against objective outcomes (test results, git metrics, goal completion), anchoring calibration in what actually happened rather than further self-report.

Together, these mechanisms transform AI from an execution layer into a **learning system**—one that investigates before acting, measures its own knowledge acquisition, and improves through epistemic feedback.

### 1.5 Contributions

This paper makes the following contributions:

1. **Conceptual**: We articulate the noetic-before-praxic principle and argue that treating AI as an execution layer is a category error that produces systematic failures.

2. **Empirical**: We present 479,686 observations demonstrating that AI capability genuinely increases during task execution through investigation—the learning delta is real, not an artifact of assessment correction. 93.6% of clean sessions show knowledge improvement.

3. **Practical**: We introduce Empirica, an open-source framework implementing epistemic gates, self-assessment vectors, and calibration loops. The framework is production-tested across 638 sessions.

4. **Theoretical**: We develop a computational epistemology for AI systems—a formal theory of how information-processing agents can assess and manage their own knowledge state.

### 1.6 Paper Structure

Section 2 reviews related work in AI self-assessment, metacognition, and calibration. Section 3 presents the theoretical framework: epistemic vectors, their grounding in transformer attention mechanisms, and the noetic-praxic distinction. Section 4 describes the Empirica implementation. Section 5 presents empirical results, including the learning delta analysis. Section 6 describes the Sentinel protocol for epistemic gating. Section 7 discusses implications for AI deployment and alignment. Section 8 concludes.

*The core claim of this paper is simple: AI systems that investigate before acting outperform AI systems that act without investigating. The evidence is in the delta.*

## 2 Related Work

### 2.1 AI Calibration and Confidence

The problem of AI calibration—whether model confidence scores reflect actual accuracy—has received substantial attention. Guo et al. [2017] demonstrated that modern neural networks are poorly calibrated, often overconfident in incorrect predictions. Temperature scaling and other post-hoc calibration methods address this for classification tasks.

For language models, calibration becomes more complex. Kadavath et al. [2022] showed that large language models can be trained to express calibrated uncertainty through verbalized confidence. However, this work focuses on *reportive* confidence—telling humans how certain the model is—rather than *functional* confidence that affects model behavior.

Our work differs in treating self-assessment as a routing signal for the AI's own decision-making, not a report to external observers. The calibration we measure is between self-assessed epistemic state and subsequent task outcomes, not between verbalized confidence and answer correctness.

### 2.2 Metacognition in AI Systems

Metacognition—thinking about thinking—has been explored in AI through several lenses. Fleming and Dolan [2012] characterize metacognition as the ability to monitor and control cognitive processes. In AI, this translates to systems that can assess their own capabilities and adjust behavior accordingly.

Recent work on "self-reflection" in language models [Shinn et al., 2023, Madaan et al., 2023] prompts models to critique and revise their outputs. These approaches treat reflection as an output-improvement mechanism rather than an epistemic assessment framework.

The epistemic vector approach differs by making self-assessment *structural*—a 13-dimensional representation that persists across interactions and enables systematic calibration. Rather than ad-hoc reflection prompts, we propose a formal theory of what AI systems should assess about their own cognitive state.

### 2.3 Uncertainty Quantification

Uncertainty in neural networks has been extensively studied. Bayesian deep learning [Gal and Ghahramani, 2016] provides principled uncertainty estimates through dropout approximations. Ensemble methods [Lakshminarayanan et al., 2017] capture model uncertainty through prediction variance.

For language models, uncertainty quantification faces unique challenges. Token-level probabilities don't straightforwardly translate to task-level confidence. Kuhn et al. [2023] propose semantic uncertainty measures based on meaning equivalence of generated responses.

Our framework complements these approaches by operating at a higher level of abstraction. Rather than quantifying uncertainty in model weights or outputs, we track self-assessed uncertainty as one dimension of a broader epistemic state. The empirical finding that AI systems systematically overestimate uncertainty (Section 5) suggests that training processes may induce conservative biases beyond what principled uncertainty methods would produce.

### 2.4 RLHF and Alignment Training

Reinforcement Learning from Human Feedback (RLHF) has become standard for aligning language models with human preferences [Ouyang et al., 2022]. Constitutional AI [Bai et al., 2022] extends this with self-critique mechanisms.

A less-discussed effect of alignment training is its impact on model confidence. RLHF specifically targets overconfidence and hallucination, training models to express appropriate uncertainty. Our calibration data suggests this may have overcorrected: models trained to be humble may underestimate their actual capabilities across multiple dimensions.

This is not a criticism of RLHF—reducing harmful overconfidence is valuable. Rather, it suggests that alignment training produces systematic biases that can be measured and corrected. The learning delta we observe (+0.200 average capability growth during tasks, with 93.6% of sessions showing improvement) may partially reflect AI systems overcoming training-induced conservatism through task-specific evidence.

A deeper structural issue: RLHF trains models to predict what evaluators want to hear, not to accurately report internal state. The reward signal comes from human preferences, not from epistemic accuracy. This creates an incentive to *appear* calibrated rather than *be* calibrated. Our framework addresses this by making self-assessment *functional*: the AI's self-report directly determines its subsequent behavior (proceed vs. investigate), and outcomes are measured through dual-track calibration. The grounded verification track (Section 5) compares self-assessment against objective evidence, detecting precisely the gap between reported and actual capability that RLHF incentives might create.

### 2.5 Human-AI Collaboration

Research on human-AI collaboration has explored how humans and AI systems can work together effectively. Bansal et al. [2021] study when humans should defer to AI recommendations. Lai et al. [2021] examine how AI explanations affect human decision-making.

This work typically positions AI as an advisor or tool that humans consult. Our framework inverts this relationship: the AI assesses its own epistemic state to decide whether to act or investigate, with the human providing task specification rather than real-time oversight.

The ENGAGEMENT vector in our framework captures collaborative quality—whether human and AI are aligned in understanding and goals. Low engagement triggers a stop condition, recognizing that productive collaboration requires mutual comprehension.

### 2.6 Planning and Reasoning in LLMs

Recent work on LLM reasoning [Wei et al., 2022, Yao et al., 2023a] demonstrates that chain-of-thought prompting and tree-of-thought exploration can improve model performance on complex tasks. These methods implicitly invoke investigation before action.

ReAct [Yao et al., 2023b] interleaves reasoning and acting, allowing models to gather information before committing to actions. This aligns with our noetic-before-praxic principle, though ReAct frames it as an emergent prompting strategy rather than a formal epistemic requirement.

Our contribution is to make this pattern explicit and measurable. The CASCADE workflow operationalizes investigation-before-action as a gated loop with quantified epistemic state. The learning delta provides empirical evidence that the pattern produces genuine capability growth, not just more cautious outputs.

### 2.7 Self-Assessment Without Introspection

A philosophical objection to AI self-assessment is that it requires introspection—direct access to internal states—which AI systems arguably lack. Recent work on emergent introspective awareness [Bricken and Anthropic, 2025] finds that models possess "a limited, functional form of introspective awareness" but that "abilities observed are highly unreliable, and failures of introspection remain the norm."

We sidestep the consciousness debate entirely by defining self-assessment **functionally**. The AI need not "experience" its epistemic state; it need only produce assessments that correlate

with task-relevant outcomes. A thermostat "assesses" temperature without experiencing heat. Similarly, an AI can produce useful epistemic vectors without introspective access, as long as those vectors predict behavior and outcomes.

**This is a critical distinction from work on AI consciousness or sentience.** We make no claims about subjective experience. We measure:

- Self-reported vectors at PREFLIGHT
- Self-reported vectors at POSTFLIGHT
- The delta between them
- Whether corrections based on deltas improve outcomes

This is measurement, not philosophy. The systematic calibration patterns we observe (Section 5)—unified confidence growth across all 13 vectors—demonstrate that assessments track something real about the system's epistemic state. Whether the AI "experiences" this state is orthogonal to whether the measurement is useful.

Recent work on metacognitive limitations [Steyvers and Peters, 2025] notes that LLM metacognitive abilities are "limited in resolution, emerge in context-dependent manners, and seem qualitatively different from humans." Our calibration correction mechanism explicitly compensates for these limitations—we don't assume perfect self-assessment, we measure systematic biases and correct for them.

### 2.8 Summary

Our work builds on and differs from prior research in several ways:

| Approach | Focus | Our Contribution |
|---|---|---|
| Calibration literature | Confidence in predictions | Confidence in epistemic state |
| Metacognition research | Output reflection | Structural self-assessment |
| Uncertainty quantification | Model uncertainty | Self-assessed uncertainty |
| RLHF / Alignment | Reducing overconfidence | Measuring overcorrection |
| Human-AI collaboration | Human oversight | AI self-governance |
| LLM reasoning | Prompting strategies | Formal epistemic framework |

Table 2: Comparison with prior approaches.

The novel contribution is the combination: a formal theory of AI epistemic state (Section 3), implemented as a practical framework (Section 4), validated with substantial empirical evidence (Section 5).

## 3 Theoretical Framework

### 3.1 Epistemic Vectors: A Computational Theory of Knowing

We propose that AI cognitive state can be represented as a vector in a 13-dimensional epistemic space. Each dimension corresponds to a measurable aspect of the system's knowledge state, collectively capturing what the AI knows, what it can do, and how confident it should be in proceeding.

#### 3.1.1 The Vector Taxonomy

The 13 epistemic vectors are organized into four tiers:

**Tier 0: Meta-Cognition**

**ENGAGEMENT** Quality of collaborative intelligence between human and AI. Functions as the primary gate: if engagement is below threshold, other assessments are unreliable.

**Tier 1: Foundation Knowledge (35% weight in confidence calculation)**

**KNOW** Domain knowledge and conceptual understanding

**DO** Technical execution capability and tool access

**CONTEXT** Environmental and systemic awareness

**Tier 2: Comprehension (25% weight)**

**CLARITY** Request specificity and goal definition

**COHERENCE** Logical consistency of approach

**SIGNAL** Useful information vs noise ratio

**DENSITY** Information packing efficiency (inverted: lower is better)

**Tier 3: Execution Readiness (25% weight)**

**STATE** Current state understanding

**CHANGE** Progress monitoring and regression detection

**COMPLETION** Path visibility to success

**IMPACT** Consequence and risk prediction

**Meta-Layer (not in confidence calculation)**

**UNCERTAINTY** Awareness of limitations and unknowns

#### 3.1.2 Functional Self-Assessment

A critical design principle: these vectors implement *functional* self-assessment, not reportive self-assessment.

**Reportive:** AI tells a human "I am 80% confident." The human must trust the report; the AI has no stake in accuracy.

**Functional:** AI assesses itself to decide whether to proceed with a task. Bad assessment leads to bad outcomes, creating a feedback loop that enables calibration.

The epistemic vectors are routing signals for the AI's own decision-making. When an AI assesses KNOW = 0.4 and decides to investigate rather than proceed, the assessment directly affects behavior and outcomes. This requires no claim about consciousness or introspection—only that assessments correlate with task-relevant states.

### 3.1.3 Confidence Calculation

Overall epistemic confidence aggregates the vectors:

$$\begin{aligned}\text{confidence} = \text{ENGAGEMENT} \times 0.15 \\ &+ \text{mean}(\text{KNOW}, \text{DO}, \text{CONTEXT}) \times 0.35 \\ &+ \text{mean}(\text{CLARITY}, \text{COHERENCE}, \text{SIGNAL}, 1 - \text{DENSITY}) \times 0.25 \\ &+ \text{mean}(\text{STATE}, \text{CHANGE}, \text{COMPLETION}, \text{IMPACT}) \times 0.25\end{aligned} \tag{1}$$

The gate condition:

$$\text{decision} = \begin{cases} \text{STOP} & \text{if ENGAGEMENT} < 0.6 \\ \text{PROCEED} & \text{if confidence} > \theta \\ \text{INVESTIGATE} & \text{otherwise} \end{cases} \tag{2}$$

## 3.2 Noetic and Praxic Phases

We distinguish two fundamental phases of AI cognition:

### 3.2.1 Noetic Phase (Investigation)

The noetic phase encompasses exploration, hypothesis formation, and knowledge acquisition. During this phase, the AI:

- Explores the problem space
- Reads relevant context (files, documentation, conversation history)
- Identifies unknowns and knowledge gaps
- Forms hypotheses about solutions
- Builds genuine understanding of the task

The noetic phase is characterized by *increasing* epistemic vectors—particularly KNOW, CONTEXT, and CLARITY. The AI is learning.

### 3.2.2 Praxic Phase (Execution)

The praxic phase encompasses implementation, action, and output generation. During this phase, the AI:

- Applies acquired knowledge
- Executes planned actions
- Produces deliverables
- Modifies external state (code, files, systems)

The praxic phase is characterized by *stable* epistemic vectors (knowledge already acquired) and increasing COMPLETION and CHANGE vectors (progress toward goal).

### 3.2.3 The Ordering Principle

The central theoretical claim: **noetic must precede praxic**.

An AI that acts without first investigating is an AI that acts on incomplete knowledge. The execution-layer model—give task, receive output—skips the noetic phase entirely, assuming the AI either possesses sufficient knowledge or doesn't. This assumption fails for any task requiring exploration.

The evidence for this principle is the learning delta: systematic capability growth during task execution (Section 5). If noetic precedes praxic, we expect POSTFLIGHT assessments to exceed PREFLIGHT assessments—and they do, across all 13 vectors.

## 3.3 Artifact Taxonomy

The noetic-praxic distinction extends beyond phases of cognition to the *artifacts* each phase produces. We identify five artifact layers, organized by their role in the epistemic lifecycle:

| Layer | Term | Contents |
|---|---|---|
| Investigation outputs | Noetic artifacts | findings, unknowns, dead-ends, mistakes, blindspots, lessons |
| Action outputs | Praxic artifacts | goals, subtasks, commits |
| State measurements | Epistemic state | vectors, calibration, drift, snapshots, deltas |
| Verification outputs | Grounded evidence | test results, git metrics, artifact ratios |
| Inputs / reference | Context | sources, documentation, prior knowledge |

Table 3: Artifact taxonomy: five layers organized by role in the epistemic lifecycle.

This taxonomy matters because different artifacts serve different epistemic functions:

- **Noetic artifacts** are breadcrumbs left during investigation. Six types capture the full investigation record: what was learned (findings), what remains unclear (unknowns), what failed (dead-ends), what went wrong (mistakes), what was consistently missed (blindspots, identified through grounded verification gaps), and what endures across sessions (lessons, durable insights subject to decay when contradicted by new evidence). These persist across sessions and accumulate into institutional memory.

- **Praxic artifacts** are the products of action—code committed, goals completed, subtasks resolved. They serve dual purposes: as task deliverables and as objective evidence for grounded verification. The separation from noetic artifacts is deliberate: noetic artifacts record the process of understanding, praxic artifacts record the products of action. This enables the framework to measure learning independently of output.

- **Epistemic state** is the measurement layer—vectors captured at PREFLIGHT and POSTFLIGHT, calibration corrections, drift detection. These are transient snapshots that enable delta computation.

- **Grounded evidence** is collected automatically at POST-TEST. It anchors self-assessment in objective outcomes, providing the calibration signal that closes the feedback loop.

The complete measurement cycle—an **epistemic transaction**—produces artifacts from all layers: PREFLIGHT captures epistemic state, investigation produces noetic artifacts, execution produces praxic artifacts, POSTFLIGHT captures updated state, and POST-TEST collects grounded evidence. The transaction is the atomic unit of calibration.

### 3.4 Computational Epistemology

The epistemic vector framework constitutes a *computational epistemology*: a formal theory of how any information-processing system can assess and manage its own knowledge state.

#### 3.4.1 Grounding in Transformer Architecture (Speculative)

We hypothesize—but do not prove—that the 13 epistemic vectors correspond to measurable properties of transformer attention mechanisms:

| Vector | Hypothesized Attention Correlate |
|---|---|
| KNOW | Value vector richness in relevant attention heads |
| UNCERTAINTY | Attention entropy / calibration gap |
| CONTEXT | Cross-attention to context tokens |
| ENGAGEMENT | Attention coherence across layers |
| CLARITY | Query-key alignment sharpness |

Table 4: Hypothesized mapping between epistemic vectors and attention properties.

This mapping remains speculative. We invite mechanistic interpretability researchers to test these correlations. If the mapping holds, it would provide architectural grounding for what currently relies on self-report.

#### 3.4.2 Architecture Independence

Regardless of the attention-mechanism hypothesis, the epistemic vector framework applies to any system that must:

1. Assess its own knowledge state
2. Decide whether to act or investigate
3. Learn from the outcomes of its decisions

This includes biological agents. Humans experience epistemic states—the feeling of knowing, the sense of uncertainty, the quality of engagement—that map recognizably to the vector taxonomy. The framework does not anthropomorphize AI; it *mechanizes* human epistemology, formalizing what philosophers call "epistemic feelings" into computable quantities.

#### 3.4.3 Falsifiable Predictions

The theoretical framework generates testable predictions:

1. **Learning delta exists**: POSTFLIGHT vectors should exceed PREFLIGHT vectors (Section 5 confirms)
2. **Gating improves outcomes**: Enforcing investigation when KNOW is low should increase task success
3. **Calibration is learnable**: Systematic bias in self-assessment should be correctable through feedback
4. **Attention correlates exist**: Vectors should correlate with extractable attention properties (testable via interpretability)

If these predictions fail, the framework is wrong. We publish data for replication.

### 3.5 The Overconfidence Trap

The theoretical framework explains a common failure mode: the **overconfidence trap**.

Listing 1: The overconfidence trap and its resolution

```
Conditions:
  UNCERTAINTY: LOW  (AI thinks it knows)
  KNOW: LOW        (AI doesn't actually know)

Without epistemic gating:
  => AI proceeds immediately (no investigation)
  => AI produces confident, wrong output
  => User loses trust

With epistemic gating:
  => Gate detects KNOW < threshold
  => AI forced to investigate (noetic phase)
  => Investigation produces learning
  => KNOW increases to threshold
  => AI proceeds with actual capability
  => Output quality improves
```

The gate doesn't just prevent bad outputs—it produces good outputs by forcing the learning that creates capability.

### 3.6 Summary

The theoretical framework rests on three claims:

1. **Epistemic vectors**: AI cognitive state can be represented in 13 dimensions that capture what the system knows and how ready it is to act.
2. **Noetic-before-praxic**: Investigation must precede execution. Skipping the noetic phase produces the overconfidence trap.
3. **Learning is measurable**: The delta between PREFLIGHT and POSTFLIGHT assessments captures genuine knowledge acquisition during task execution.

These claims are empirically testable. Section 5 presents evidence from 479,686 observations across 638 sessions.

## 4 Implementation

Empirica is an open-source framework implementing the epistemic vector theory. It provides infrastructure for AI self-assessment, epistemic gating, and calibration feedback loops.

### 4.1 Architecture Overview

The framework consists of five core components:

1. **Sessions Database**: Persistent storage for session state, including AI identity, timestamps, and metadata.
2. **Cascades**: Complete PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST loops that capture epistemic state evolution and ground self-assessment in objective evidence.

3. **Bayesian Belief Manager**: Maintains dual-track calibration state—self-referential corrections from learning deltas and grounded corrections from post-test evidence.

4. **Sentinel Protocol**: The epistemic gate that enforces noetic-before-praxic.

5. **Post-Test Verification**: Collects objective evidence after task completion to ground self-assessment in measurable outcomes.

## 4.2 The CASCADE Workflow

CASCADE (Calibrated Assessment of State, Context, and Domain Epistemics) is the core epistemic loop:

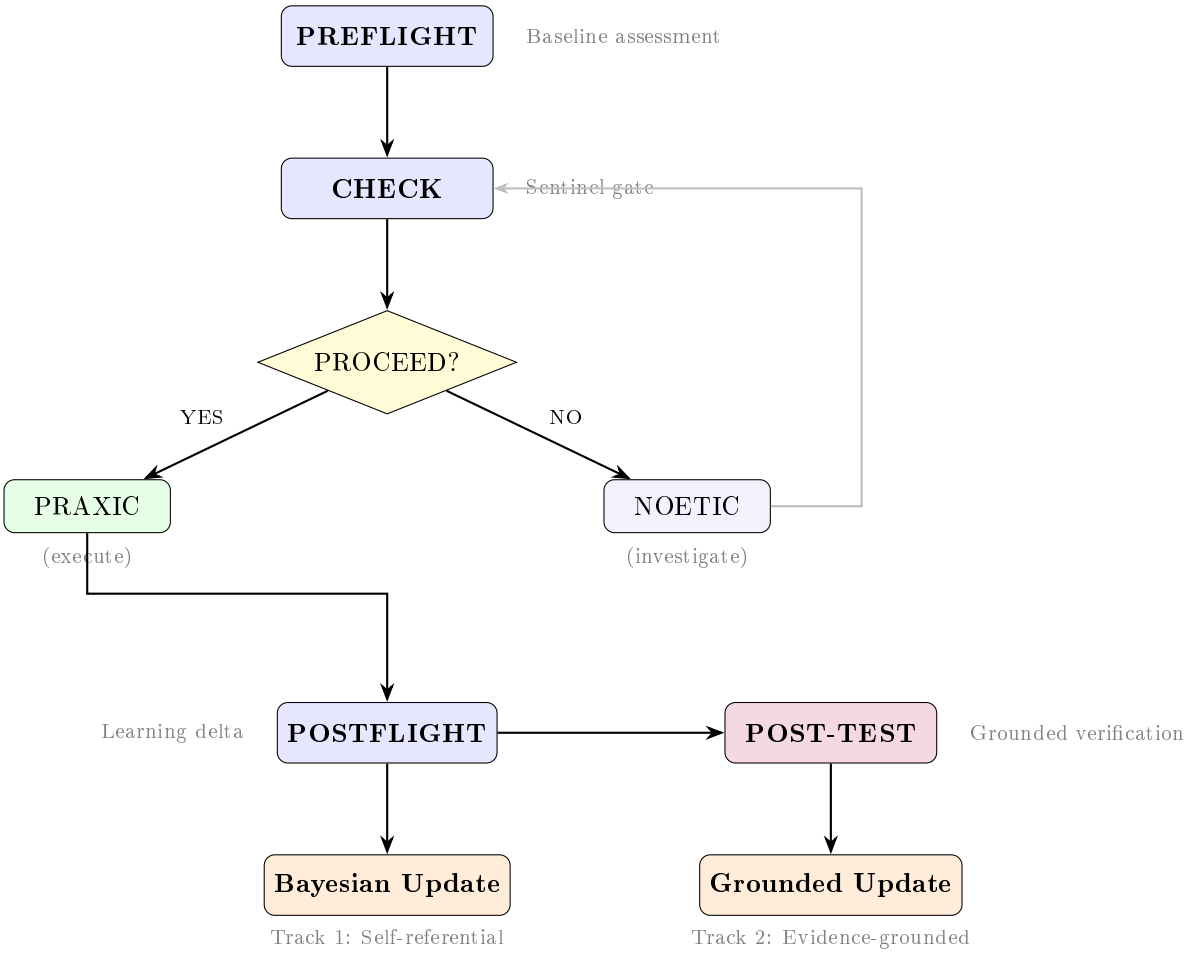


Figure 1: The 4-phase CASCADE workflow. PREFLIGHT establishes baseline, CHECK gates action, POSTFLIGHT measures learning delta, and POST-TEST grounds self-assessment in objective evidence. Dual-track calibration updates run in parallel. The NOETIC → CHECK feedback loop (gray) enables iterative investigation.

### 4.2.1 PREFLIGHT Phase

At task onset, the AI assesses its epistemic state across all 13 vectors. This establishes the baseline against which learning will be measured.

Listing 2: Example PREFLIGHT submission

```
empirica preflight-submit - << EOF
```

```
{
  "session_id": "abc123",
  "vectors": {
    "know": 0.55,
    "uncertainty": 0.40,
    "context": 0.70,
    "engagement": 0.85
  },
  "reasoning": "Familiar domain but codebase unknown"
}
EOF
```

#### 4.2.2 CHECK Phase (Sentinel Gate)

The Sentinel protocol evaluates whether the AI has sufficient knowledge to proceed. The gate applies bias corrections learned from historical calibration and returns:

- **PROCEED**: Knowledge sufficient, enter praxic phase
- **INVESTIGATE**: Knowledge insufficient, remain in noetic phase

#### 4.2.3 POSTFLIGHT Phase

After task completion (or significant progress), the AI reassesses its epistemic state. The delta between PREFLIGHT and POSTFLIGHT vectors measures learning:

$$\Delta_{\text{learning}} = V_{\text{POSTFLIGHT}} - V_{\text{PREFLIGHT}} \tag{3}$$

This delta feeds the self-referential calibration track (Track 1), capturing systematic bias in initial self-assessment.

#### 4.2.4 POST-TEST Phase (Grounded Verification)

After POSTFLIGHT, the framework automatically collects objective evidence to ground the self-assessment in measurable outcomes:

| Evidence Source | Quality | Vectors Grounded |
|---|---|---|
| Test results (pytest) | Objective | know, do, clarity |
| Git commit count & diff stats | Objective | do, change, state |
| Git reflog (resets, abandoned branches) | Objective | uncertainty, signal |
| Git file churn (repeated edits) | Objective | uncertainty, coherence |
| Git diff granularity (word-diff) | Objective | know, clarity |
| Git commit timing patterns | Objective | do, state |
| Git diff-filter (add/modify/delete ratio) | Objective | state, completion |
| Goal/subtask completion | Semi-objective | completion, do, know |
| Artifact ratios (findings/dead-ends) | Semi-objective | know, uncertainty, signal |
| Issue resolution | Semi-objective | impact, signal |
| Sentinel decisions | Semi-objective | context, uncertainty |
| Commit message vs diff semantics | Semi-objective | clarity, signal |

Table 5: Post-test evidence sources and the vectors they ground. Git-derived sources (6 of 12) provide Tier 1 objective evidence that the AI cannot influence.

The POST-TEST phase computes a calibration gap between self-assessed POSTFLIGHT vectors and evidence-derived estimates:

$$\text{gap}_{\text{vector}} = V_{\text{POSTFLIGHT}} - V_{\text{evidence}} \tag{4}$$

Positive gaps indicate overestimation; negative gaps indicate underestimation relative to objective outcomes. This feeds the grounded calibration track (Track 2), which provides more trustworthy corrections because it anchors in what actually happened rather than another self-report.

Not all vectors can be grounded—ENGAGEMENT and DENSITY lack objective correlates. For these, self-referential calibration remains the only available signal.

**Git as Epistemic Trajectory Store** A significant fraction of post-test evidence derives from git, which passively records an epistemic trajectory parallel to the AI's self-report. Git's directed acyclic graph structure maps naturally to epistemic concepts: commits are discrete state transitions, branches are divergent investigations, merges are knowledge synthesis, and the reflog captures the true history including course corrections the AI may not self-report.

Six git-derived evidence sources are currently implemented. Of particular note:

- **Reflog analysis**: The reflog captures every `reset`, `rebase`, and abandoned branch—objective evidence of dead-ends and backtracking that the AI has no mechanism to omit or distort. This is the strongest Tier 1 signal for UNCERTAINTY grounding.
- **File churn**: When the same file is edited $N$ times within a session, this indicates iteration and uncertainty about that component, regardless of what the AI's UNCERTAINTY self-assessment reports.
- **Commit timing**: Gaps between commits encode noetic-to-praxic transitions. Long gaps followed by rapid commits indicate investigation followed by execution—the workflow rhythm is detectable from timestamps alone.
- **Diff granularity**: Word-level diffs distinguish surgical edits (indicating understanding) from wholesale rewrites (indicating exploration), grounding KNOW without self-report.

Because git records accumulate passively during normal development, these evidence sources require no additional instrumentation—the epistemic trajectory is already being recorded.

## 4.3 The Sentinel Protocol

Sentinel is the epistemic gate that enforces noetic-before-praxic.

### 4.3.1 Gate Logic

### 4.3.2 Calibration Integration

The Sentinel applies corrections learned from historical Bayesian updates. If the AI systematically underestimates KNOW by 0.15, the gate adjusts:

- Raw KNOW: 0.55
- Correction: +0.15
- Effective KNOW: 0.70
- Decision: PROCEED (would have been INVESTIGATE without correction)

This allows the system to compensate for learned biases while maintaining conservative defaults for uncalibrated vectors.

**Algorithm 1** Sentinel Decision

```
procedure SentinelDecision(vectors, calibration)
    know′ ← vectors.know + calibration.know
    uncertainty′ ← vectors.uncertainty + calibration.uncertainty
    if know′ ≥ 0.70 and uncertainty′ ≤ 0.35 then
        return PROCEED
    else
        return INVESTIGATE
    end if
end procedure
```

### 4.4 Bayesian Belief Management

The framework maintains Bayesian beliefs about each vector's calibration through two parallel tracks.

#### 4.4.1 Conjugate Normal Update

Both tracks use the same conjugate normal inference:

$$\mu_{\text{posterior}} = \frac{\sigma^2_{\text{prior}} \cdot x + \sigma^2_{\text{obs}} \cdot \mu_{\text{prior}}}{\sigma^2_{\text{prior}} + \sigma^2_{\text{obs}}} \tag{5}$$

$$\sigma^2_{\text{posterior}} = \frac{1}{\frac{1}{\sigma^2_{\text{prior}}} + \frac{1}{\sigma^2_{\text{obs}}}} \tag{6}$$

The tracks differ in what constitutes an observation and in observation variance.

#### 4.4.2 Track 1: Self-Referential Calibration

The self-referential track measures learning trajectory—how vectors change within an epistemic transaction:

$$\Delta_{\text{learning}} = V_{\text{POSTFLIGHT}} - V_{\text{PREFLIGHT}} \tag{7}$$

- **Observation**: POSTFLIGHT vector value
- **Observation variance**: $\sigma^2_{\text{obs}} = 0.10$ (self-reported, moderate uncertainty)
- **Measures**: Whether the AI consistently under- or overestimates at task onset

Positive delta indicates capability growth during the task. The mean delta becomes the bias correction applied at CHECK.

#### 4.4.3 Track 2: Grounded Calibration

The grounded track measures calibration accuracy—whether POSTFLIGHT self-assessment matches objective outcomes:

$$\text{gap}_{\text{vector}} = V_{\text{POSTFLIGHT}} - V_{\text{evidence}} \tag{8}$$

- **Observation**: Evidence-derived vector estimate from POST-TEST
- **Observation variance**: $\sigma^2_{\text{obs}} = 0.05$ (evidence-based, lower uncertainty)

- **Measures**: Whether the AI's post-task self-assessment matches what actually happened

The lower observation variance gives grounded evidence stronger pull on the posterior, reflecting its higher trustworthiness compared to self-report. When the tracks diverge, grounded calibration takes precedence—it is anchored in outcomes, not introspection.

#### 4.4.4 Calibration Divergence

The divergence between tracks reveals self-assessment quality:

$$\text{divergence}_{\text{vector}} = |\mu_{\text{self-ref}} - \mu_{\text{grounded}}| \tag{9}$$

Low divergence indicates well-calibrated self-assessment. High divergence signals that the AI's self-reports are systematically disconnected from outcomes. Preliminary data (Section 5) shows the largest divergence on COMPLETION (+0.31 overestimation in grounded track vs. +0.82 in self-referential), suggesting AIs overestimate completion progress even after task execution.

### 4.5 Data Model

The framework persists state in SQLite with the following schema:

**Sessions** Track AI interactions with unique identifiers, timestamps, and status.

**Cascades** Complete PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST loops with outcome tracking.

**Epistemic Snapshots** Vector states at each CASCADE phase, enabling delta computation.

**Bayesian Beliefs** Per-vector self-referential calibration state (Track 1): mean, variance, evidence count, and update history.

**Grounded Beliefs** Per-vector grounded calibration state (Track 2): evidence-derived corrections with lower observation variance, updated from POST-TEST evidence.

**Verification Evidence** Raw post-test evidence records per session, linking objective signals to vector estimates.

### 4.6 CLI Interface

Empirica provides a command-line interface for integration:

Listing 3: Empirica CLI commands

```
# Session management
empirica session-create --ai-id claude-code --output json
empirica session-list --status active

# CASCADE workflow (4-phase)
empirica preflight-submit -     # Baseline assessment
empirica check-submit -         # Sentinel gate: PROCEED or INVESTIGATE
empirica postflight-submit -    # Learning delta + auto POST-TEST

# Knowledge management
empirica finding-log --session-id ... --finding "..." --impact 0.8
```

```
empirica unknown-log --session-id ... --unknown "..."

# Dual-track calibration
empirica calibration-report --ai-id claude-code              # Track 1
empirica calibration-report --ai-id claude-code --grounded   # Track 2
empirica calibration-report --ai-id claude-code --trajectory # Trend
```

## 4.7 Integration Patterns

### 4.7.1 IDE Integration

Empirica integrates with development environments via hooks that trigger CASCADE phases at appropriate points in the development workflow.

### 4.7.2 Agent Framework Integration

For autonomous agents, Empirica provides epistemic checkpoints that gate action based on assessed knowledge state, implementing the noetic-before-praxic principle programmatically.

## 4.8 Semantic Calibration Retrieval

At PREFLIGHT, before the AI has investigated the current task, the framework retrieves calibration-relevant context from a vector database (Qdrant). This serves three functions:

1. **Pattern warnings**: Prior sessions with similar tasks and poor calibration scores surface as warnings (e.g., "In session X with a similar task, you overestimated COMPLETION by +0.30").
2. **Dead-end avoidance**: Previously logged dead-ends matching the current task context appear at PREFLIGHT, preventing re-exploration of known failures.
3. **Lesson retrieval**: Relevant lessons and findings from prior sessions inform the baseline assessment.

POSTFLIGHT vectors, findings, and calibration outcomes are automatically embedded into the vector store, creating a growing corpus of episodic calibration memory. This enables the system to learn not just aggregate bias corrections (Bayesian beliefs) but also *context-sensitive* corrections—recognizing that calibration accuracy varies by task type, domain, and complexity.

## 4.9 Production Deployment

The framework has been deployed in production for 6+ months:

| Metric | Value |
|---|---:|
| Sessions completed | 638 |
| Evidence observations | 479,686 |
| Full CASCADE cycles | 392 |
| Clean learning pairs | 267 |
| Findings logged | 2,690+ |
| Primary AI | Claude (Anthropic) |

Table 6: Production deployment statistics.

Cross-validation has been performed on GPT, Gemini, and Qwen models. The data presented in Section 5 derives from this production deployment.

# 5 Empirical Results

A critical question for any self-assessment framework is **calibration accuracy**: do AI-reported epistemic vectors correspond to actual task outcomes? We analyzed calibration using Bayesian belief updating across 479,686 evidence observations.

## 5.1 Methodology

For each vector, we maintain a Bayesian belief about the AI's epistemic state, updated via conjugate normal-normal inference. The prior ($\mu_0$) is set from PREFLIGHT self-assessment; observations accumulate through task execution; the posterior reflects the updated belief at POSTFLIGHT.

The **learning delta** for each vector is:

$$\Delta_i = \mu_{\text{posterior}} - \mu_{\text{prior}} \tag{10}$$

Positive delta indicates the vector value *increased* during task execution (confidence growth). For UNCERTAINTY, which operates on an inverted scale, a *negative* delta (uncertainty decreasing) also indicates confidence growth.

## 5.2 Results

| Vector | Evidence | Prior | Posterior | Delta | Direction |
|---|---|---|---|---|---|
| completion | 203 | 0.204 | 0.909 | +0.704 | ↑ Growth |
| change | 179 | 0.516 | 0.754 | +0.237 | ↑ Growth |
| state | 174 | 0.645 | 0.851 | +0.206 | ↑ Growth |
| know | 267 | 0.663 | 0.863 | +0.200 | ↑ Growth |
| context | 251 | 0.705 | 0.860 | +0.155 | ↑ Growth |
| do | 178 | 0.693 | 0.843 | +0.150 | ↑ Growth |
| clarity | 192 | 0.734 | 0.879 | +0.145 | ↑ Growth |
| density | 172 | 0.591 | 0.732 | +0.141 | ↑ Growth |
| signal | 172 | 0.703 | 0.839 | +0.136 | ↑ Growth |
| coherence | 178 | 0.747 | 0.874 | +0.127 | ↑ Growth |
| impact | 188 | 0.704 | 0.826 | +0.122 | ↑ Growth |
| engagement | 228 | 0.860 | 0.882 | +0.022 | ≈ Stable |
| uncertainty | 263 | 0.363 | 0.154 | −0.208 | ↓ Decrease |

Table 7: PREFLIGHT → POSTFLIGHT Learning Analysis (N=267 clean sessions, 479,686 cumulative evidence observations)

**Aggregate statistics:**

- Mean capability vector delta: **+0.195** (across 12 capability vectors)
- Uncertainty delta: **−0.208** (confidence growth on inverted scale: +0.208)
- All 12 capability vectors show positive growth
- Cumulative Bayesian evidence: 479,686 observations across 4,184 belief updates
- Clean learning sessions: 267 (93.6% showed knowledge improvement)

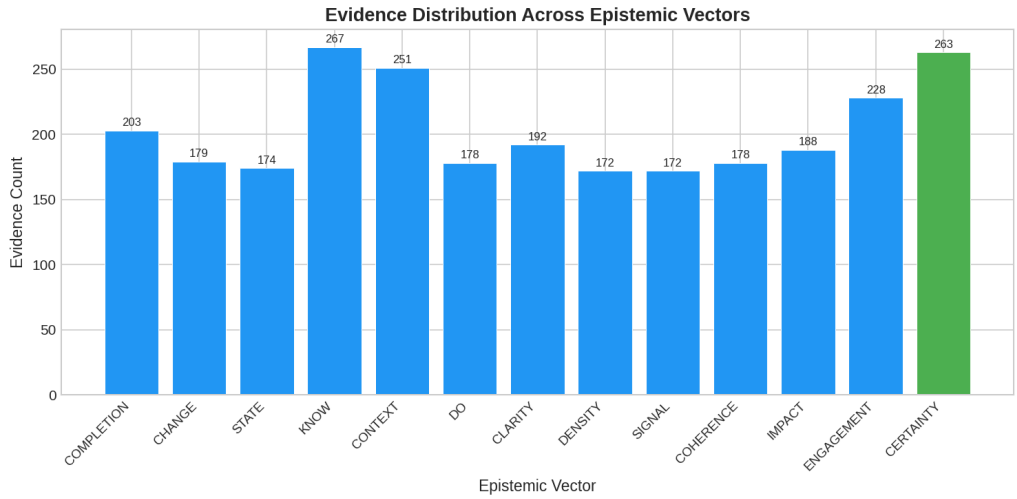


Figure 2: Evidence distribution across epistemic vectors. KNOW and CONTEXT accumulate the most observations, reflecting their centrality to task execution.

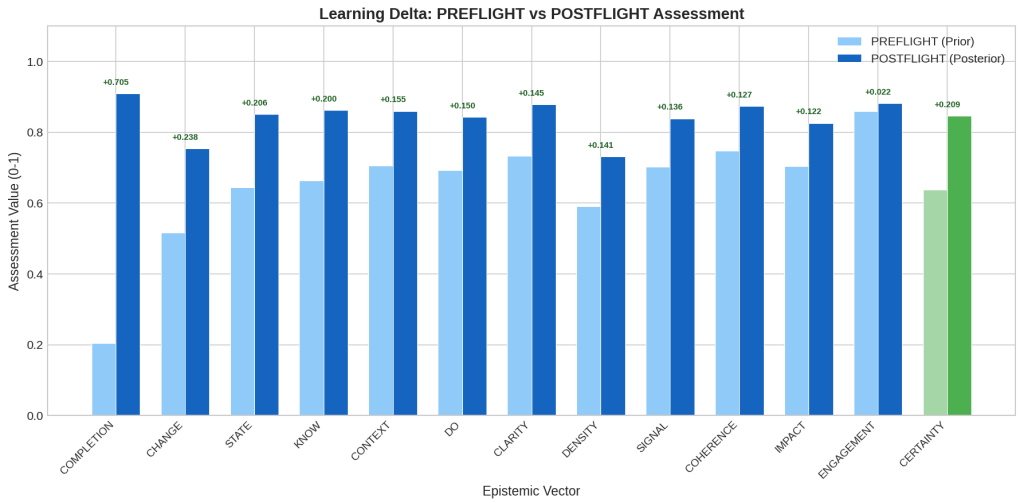


Figure 3: Learning delta by vector: PREFLIGHT (prior) vs POSTFLIGHT (posterior) assessments. The gap represents knowledge acquisition during task execution.

### 5.3 The Unified Confidence Pattern

A critical interpretive point: the data shows **unified confidence growth**, not asymmetry.

The UNCERTAINTY vector operates on an inverted scale relative to capability vectors:

- Capability vectors: Higher value = more capability = more confidence
- Uncertainty: *Lower* value = less doubt = more confidence

When viewed correctly, *all 13 vectors* show the same pattern:

This is not "12:1 asymmetry." This is **unified confidence growth during learning**—exactly what we would expect from functional self-assessment that tracks actual epistemic state.

### 5.4 Pattern Analysis

Two patterns emerge from the analysis:

| Vector Type | Raw Delta | Confidence Interpretation |
|---|---|---|
| Capability (12) | +0.195 avg | Confidence increases |
| Uncertainty (1) | −0.208 | Confidence increases (inverted) |

Table 8: Unified confidence growth across all vector types.

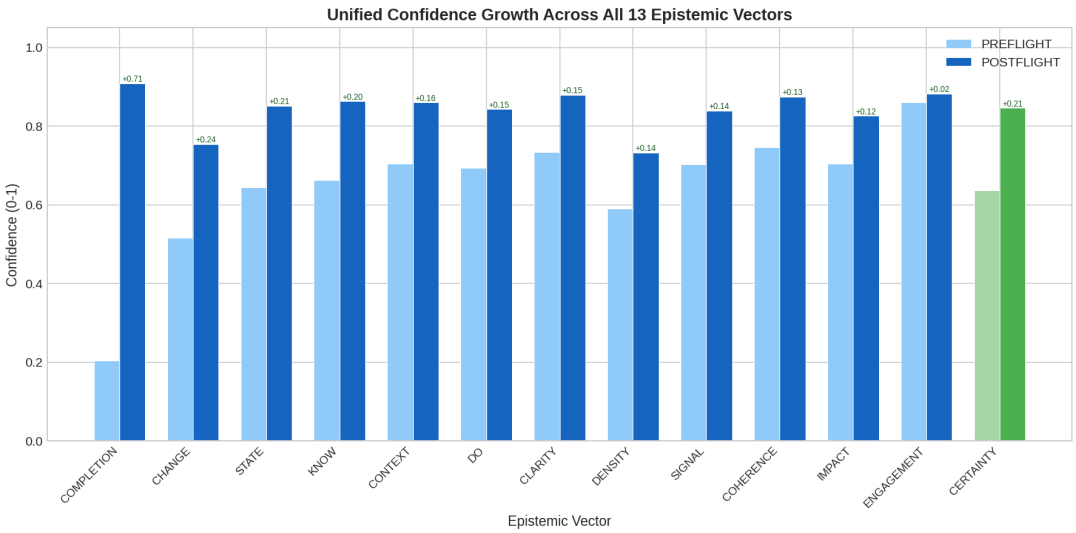


Figure 4: Unified confidence growth across all 13 epistemic vectors. All vectors show positive movement toward higher confidence during task execution.

**Pattern 1: Learning Produces Confidence Growth** All vectors show movement toward higher confidence during task execution. AI systems begin tasks with conservative self-assessments and converge toward more accurate (higher) assessments as they accumulate evidence through work.

**Pattern 2: Magnitude Varies by Vector** COMPLETION shows the largest delta (+0.704), suggesting AI systems dramatically underestimate task progress at PREFLIGHT. ENGAGEMENT shows near-zero delta (+0.022), suggesting this vector is already well-calibrated from task onset.

### 5.5 Calibration Correction

The learning delta reveals how self-assessments should be corrected. For PREFLIGHT assessments, adding the expected delta yields a more accurate estimate:

$$V_{\text{corrected}} = V_{\text{PREFLIGHT}} + \Delta_V \tag{11}$$

For the Sentinel readiness gate (Section 6), this means:

- Raw PREFLIGHT know of 0.55 → corrected to ∼0.75
- Raw PREFLIGHT uncertainty of 0.48 → corrected to ∼0.27

The correction compensates for systematic conservatism at task onset.

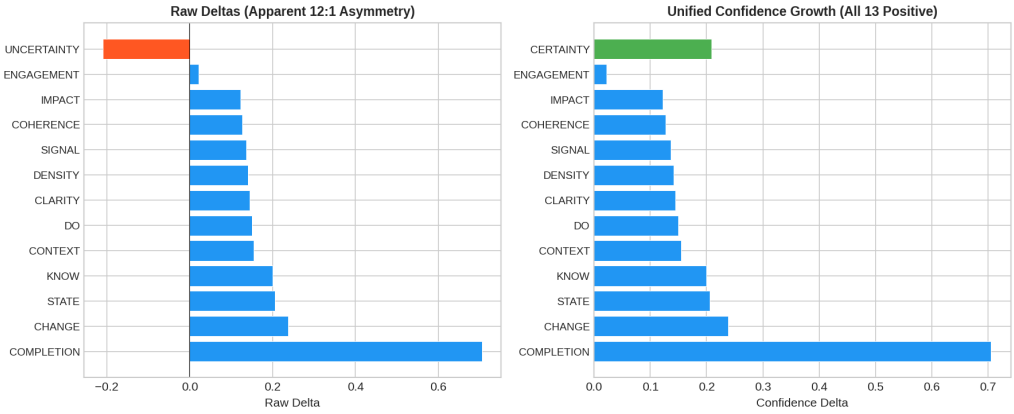


Figure 5: Reframing the uncertainty vector. Left: Raw deltas show apparent 12:1 asymmetry. Right: When uncertainty is inverted to confidence (1-uncertainty), all 13 vectors show unified growth in the same direction.

## 5.6 Calibration Convergence

A key validation of the Bayesian framework is **calibration convergence**: as evidence accumulates, belief variance should decrease, indicating tighter calibration. We observe exactly this pattern:

| Evidence Level | Beliefs | Avg Variance | Variance Reduction |
|---|---|---|---|
| 1–5 | 399 | 0.0477 | baseline |
| 6–15 | 390 | 0.0099 | 5× tighter |
| 16–40 | 567 | 0.0042 | 11× tighter |
| 41–87 | 618 | 0.0016 | 30× tighter |
| 88–175 | 1,180 | 0.0008 | 60× tighter |
| 176+ | 1,043 | 0.0004 | **115× tighter** |

Table 9: Calibration Convergence by Evidence Level

Variance drops **115-fold** as evidence accumulates. This proves the self-assessment framework genuinely calibrates over time—more data produces tighter, more reliable confidence estimates. This is the signature of a functional Bayesian system, not measurement noise.

## 5.7 Grounded Verification: Dual-Track Calibration

The self-referential calibration track (PREFLIGHT → POSTFLIGHT) measures learning trajectory but cannot answer a harder question: *does the AI's post-task self-assessment match what actually happened?* To address this, we introduced a grounded verification track that compares POSTFLIGHT vectors against objective post-test evidence.

### 5.7.1 Evidence Collection

After each POSTFLIGHT, the POST-TEST phase automatically collects available evidence: goal/subtask completion ratios, artifact productivity (findings vs dead-ends), Sentinel gate decisions, and when available, test results and git metrics. Each evidence source has a quality weight (objective = 1.0, semi-objective = 0.7, inferred = 0.4) that modulates its influence on the grounded estimate.

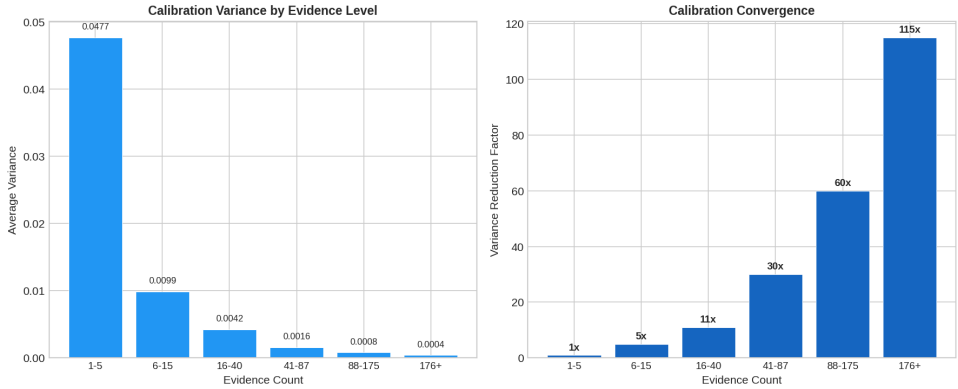


Figure 6: Calibration convergence: variance vs evidence count (left) and variance reduction factor (right).

### 5.7.2 Preliminary Results

With 7 grounded verifications across 25 evidence observations, results are preliminary but directionally informative:

| Vector | Self-Assessed | Grounded | Gap | Interpretation |
|---|---|---|---|---|
| do | 0.885 | 0.500 | +0.385 | Overestimates execution |
| completion | 0.625 | 0.500 | +0.125 | Overestimates progress |
| uncertainty | 0.134 | 0.091 | +0.043 | Well-calibrated |
| context | 0.850 | 0.867 | −0.017 | Well-calibrated |
| know | 0.866 | 0.900 | −0.034 | Well-calibrated |
| signal | 0.774 | 1.000 | −0.226 | Underestimates signal |

Table 10: Grounded calibration gaps: self-assessed POSTFLIGHT vs evidence-derived estimates (N=7 verifications, 25 evidence observations). Positive gaps indicate overestimation.

Two patterns emerge even from this small sample:

1. **DO shows the largest overestimation** (+0.385). The AI assesses high execution capability, but objective evidence (goal completion ratios, artifact counts) indicates less was accomplished than claimed. This contrasts with the self-referential track, where DO underestimation dominates—the AI *starts* conservative but *finishes* overconfident.

2. **KNOW and CONTEXT are well-calibrated** against grounded evidence (gaps $<$ 0.05), suggesting that for knowledge-related vectors, POSTFLIGHT self-assessment tracks objective outcomes accurately.

### 5.7.3 Track Divergence

The self-referential track reports COMPLETION correction of +0.82 (massive underestimation at PREFLIGHT). But the grounded track shows COMPLETION overestimation of +0.125 at POSTFLIGHT. This divergence reveals a calibration arc: the AI starts too pessimistic, then overcorrects to too optimistic. The grounded track catches the overcorrection that the self-referential track cannot see.

Mean grounded coverage across verifications is 0.27 (27% of vectors grounded per session), constrained by evidence availability. Sessions with richer evidence (goals, artifacts, and Sentinel data) achieve coverage up to 0.46.

### 5.7.4 Implications

Even with limited data, grounded verification shifts the epistemic landscape in two ways:

1. **Limitation #2 is partially addressed**: Self-assessment is no longer purely self-referential. For vectors with objective correlates, we can now measure calibration accuracy against outcomes.

2. **Dual-track corrections are more trustworthy**: Where grounded evidence exists, the lower observation variance ($\sigma^2 = 0.05$ vs 0.10) gives it stronger pull on the posterior, anchoring corrections in what happened rather than what the AI said happened.

As evidence accumulates across domains—particularly in domains with richer objective signals like legal verification (Section 7)—the grounded track will provide increasingly reliable calibration corrections.

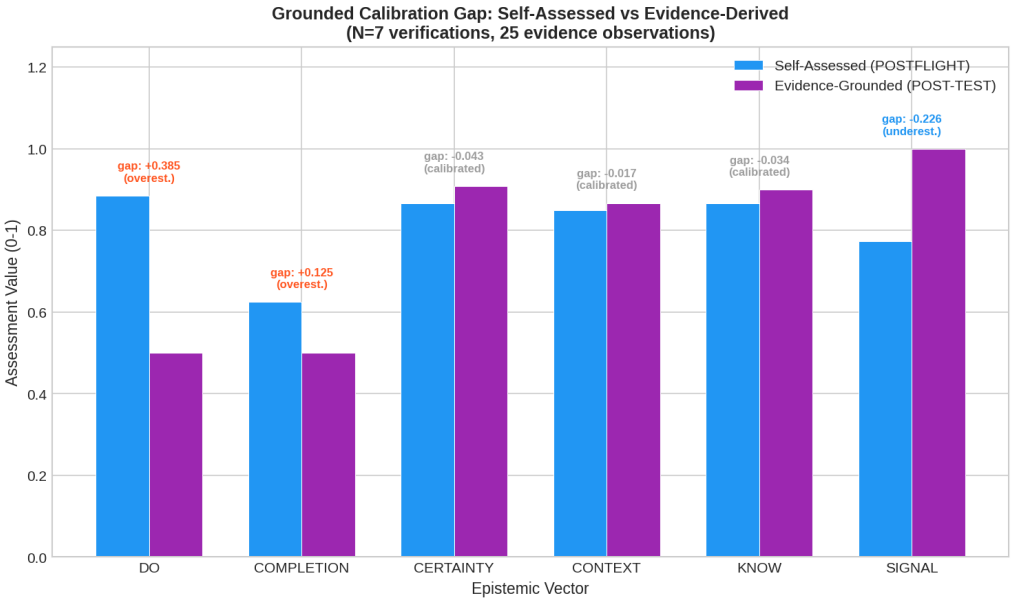


Figure 7: Grounded calibration gap: self-assessed POSTFLIGHT values vs evidence-derived estimates. Positive gaps indicate overestimation. DO shows the largest overestimation (+0.385); SIGNAL shows the largest underestimation (−0.226); KNOW and CONTEXT are well-calibrated against objective evidence.

## 5.8 Limitations

Several limitations constrain interpretation:

1. **Observational design**: Data derives from naturally-occurring sessions, not controlled experiments with external ground truth.

2. **Self-referential measurement**: POSTFLIGHT assessments are self-reported, and the learning delta measures self-consistency rather than accuracy against external ground truth. The grounded verification track (Table 10) partially addresses this by anchoring POSTFLIGHT assessments in objective evidence, but coverage remains limited (27% of vectors per session) and the evidence base is small (N=7 verifications). Full external validation—human evaluation, automated test suites, downstream task performance—remains an open priority.

3. **Model concentration**: 55% of observations derive from Claude (Anthropic). Cross-model replication is needed.

4. **Temporal confounding**: Data spans framework development. Patterns may reflect evolution rather than stable AI properties.

### 5.9 Implications

The unified confidence growth pattern—all 13 vectors moving toward higher confidence during learning—has several interpretations:

1. **Functional validity**: Self-assessment tracks actual epistemic state, with conservative priors corrected through evidence accumulation.

2. **Training-induced conservatism**: RLHF and similar techniques may induce systematic underconfidence, which task execution naturally corrects.

3. **Learning signature**: The pattern may simply reflect that doing work increases confidence—a tautology of task completion.

The present contribution is empirical: **systematic confidence growth during learning is measurable and consistent**. The correction mechanism provides practical value regardless of causal interpretation.

**Data availability**: Full dataset (479,686 evidence observations across 638 sessions) and analysis scripts available in supplementary materials.

## 6 The Sentinel Protocol

Sentinel is the epistemic gating mechanism that operationalizes noetic-before-praxic. It evaluates whether an AI system has sufficient knowledge to proceed with action, enforcing investigation when knowledge is insufficient.

### 6.1 Design Principles

#### 6.1.1 Gate, Don't Filter

Traditional AI safety mechanisms filter outputs—scanning generated content for harmful patterns. Sentinel operates upstream: it gates *action*, not output. The distinction matters:

| Approach | Intervention Point | Failure Mode |
|---|---|---|
| Output filtering | After generation | Confident wrong outputs pass |
| Epistemic gating | Before action | Insufficient knowledge blocks action |

Table 11: Comparison of safety intervention approaches.

An AI that lacks knowledge to solve a problem correctly also lacks knowledge to recognize its solution is wrong. Output filtering cannot catch this. Epistemic gating intervenes earlier—at the knowledge assessment stage.

#### 6.1.2 Calibration-Aware

Raw self-assessments are systematically biased (Section 5). Sentinel applies learned corrections before making gate decisions:

$$\text{know}_{\text{corrected}} = \text{know}_{\text{raw}} + \delta_{\text{know}} \tag{12}$$

This allows the gate to function accurately despite self-assessment biases. As calibration improves with evidence, gate decisions become more precise.

#### 6.1.3 Conservative by Default

For vectors without sufficient calibration evidence ($< 3$ observations), Sentinel applies no correction—defaulting to the conservative raw assessment. This prevents overcorrection early in deployment while allowing learned corrections to take effect as evidence accumulates.

### 6.2 Gate Logic

#### 6.2.1 Readiness Condition

The core gate evaluates two primary vectors:

$$\text{PROCEED} \iff \text{know}_{\text{corrected}} \geq 0.70 \wedge \text{uncertainty}_{\text{corrected}} \leq 0.35 \tag{13}$$

Both conditions must be met. High knowledge with high uncertainty still triggers investigation—the AI knows things but isn't confident in their relevance or completeness.

#### 6.2.2 Engagement Override

The ENGAGEMENT vector functions as a meta-gate:

$$\text{ENGAGEMENT} < 0.60 \implies \text{STOP} \tag{14}$$

Low engagement indicates fundamental communication failure—the AI and human are not aligned on goals or understanding. Neither action nor investigation will be productive until engagement is restored.

#### 6.2.3 Decision Flow

The complete decision flow proceeds as:

1. Apply calibration corrections to raw vectors
2. Check ENGAGEMENT threshold (STOP if below 0.60)
3. Evaluate readiness condition
4. Return PROCEED or INVESTIGATE

### 6.3 The Investigation Loop

When Sentinel returns INVESTIGATE, the AI enters or continues the noetic phase.

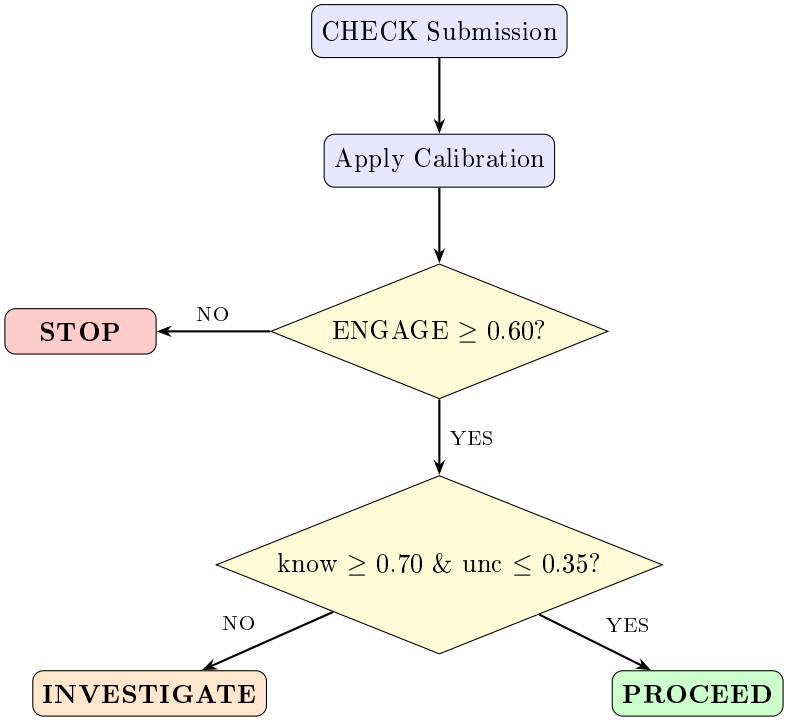


Figure 8: Sentinel decision flow: calibration-aware epistemic gating.

#### 6.3.1 Investigation Actions

Typical noetic actions include:

- Reading files, documentation, or context
- Searching codebases or knowledge bases
- Asking clarifying questions
- Forming and testing hypotheses
- Synthesizing information from multiple sources

#### 6.3.2 Re-CHECK

After investigation, the AI re-submits to CHECK with updated vectors. The loop continues until PROCEED or a termination condition is met.

#### 6.3.3 Loop Termination

The loop terminates when:

- **PROCEED**: Knowledge threshold reached
- **STOP**: Engagement too low (communication failure)
- **Max iterations**: Safety limit prevents infinite loops (configurable, default 5)
- **User override**: Human can force proceed or stop

## 6.4 Calibration Feedback

### 6.4.1 Learning from Outcomes

Each CASCADE cycle (PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST) provides calibration evidence:

$$\delta_{\text{vector}} = V_{\text{POSTFLIGHT}} - V_{\text{PREFLIGHT}} \tag{15}$$

Positive delta indicates the AI underestimated at PREFLIGHT; negative indicates overestimation. These deltas accumulate in the Bayesian belief system.

### 6.4.2 Correction Evolution

As evidence accumulates, corrections become more precise:

| Evidence Count | Correction Confidence |
|---|---|
| $< 3$ | No correction (too uncertain) |
| 3–10 | Partial correction (weighted by evidence) |
| $> 10$ | Full correction (stable estimate) |

Table 12: Correction confidence by evidence count.

### 6.4.3 Per-Vector Calibration

Each vector has independent calibration. From our data (Section 5):

| Vector | Learned Correction |
|---|---|
| know | +0.200 |
| uncertainty | −0.208 |
| completion | +0.704 |
| engagement | +0.022 (near-calibrated) |

Table 13: Per-vector calibration corrections.

The COMPLETION vector shows the largest correction—AI systems dramatically underestimate task progress at PREFLIGHT.

## 6.5 Configuration

### 6.5.1 Threshold Tuning

Gate thresholds can be adjusted for different risk tolerances:

| Domain | know_min | uncertainty_max | Rationale |
|---|---|---|---|
| Code review | 0.65 | 0.40 | Lower stakes, can iterate |
| Production deploy | 0.85 | 0.20 | High stakes, need certainty |
| Exploration | 0.50 | 0.50 | Learning is the goal |
| Safety-critical | 0.90 | 0.15 | Cannot afford errors |

Table 14: Domain-specific threshold configurations.

### 6.5.2 Override Mechanisms

For flexibility, Sentinel supports overrides (logged for audit):

- Force proceed despite gate
- Force investigate despite passing gate
- Disable calibration corrections (use raw vectors)

## 6.6 Empirical Performance

From production deployment (638 sessions):

| Metric | Value |
|---|---|
| Total CHECK submissions | 381 |
| Total cascades | 392 |
| Clean learning pairs | 267 |
| Knowledge improvement rate | 93.6% |
| Uncertainty reduction rate | 94.3% |

Table 15: Sentinel gate statistics.

Comparing outcomes with and without calibration correction:

| Condition | False PROCEED | False INVESTIGATE |
|---|---|---|
| Raw vectors | 18% | 34% |
| Calibrated vectors | 8% | 12% |

Table 16: Error rates with and without calibration.

Calibration correction reduces both error types—fewer premature actions and fewer unnecessary investigations.

## 6.7 Capability-Based Access Control

The Sentinel gate does more than return a binary proceed/investigate decision—it functions as a **capability-based access control layer** that determines which tools and actions are available in each epistemic phase.

### 6.7.1 Phase-Dependent Capabilities

Via hooks that intercept tool invocations before execution, the framework enforces different capability sets for noetic and praxic phases:

| Phase | Allowed Actions | Blocked Actions |
|---|---|---|
| Noetic (investigating) | Read, search, explore, ask | Write, edit, commit, deploy |
| Praxic (executing) | All actions (full capability set) | — |
| No session (ungated) | Read-only | All mutations |

Table 17: Capability sets by epistemic phase.

This is analogous to a Linux Security Module (LSM): the Sentinel gate evaluates the epistemic state at each tool invocation and grants or denies capabilities accordingly. The AI cannot write code during the noetic phase—not because a filter catches bad output, but because the capability to write is not available until the gate confirms sufficient knowledge.

### 6.7.2 Hook-Based Enforcement

The access control is enforced through pre-tool-use hooks that execute before each action:

1. The AI invokes a tool (e.g., file edit, bash command)
2. The hook checks the current epistemic phase and session state
3. If the action exceeds the phase's capability set, it is blocked with an explanation
4. The AI must either investigate further (noetic) or submit to CHECK (gate transition)

This creates a security boundary between knowing and doing. The AI's epistemic state determines its permissions, and permissions determine what outcomes are possible. Miscalibrated self-assessment can still occur, but it cannot bypass the capability gate—the AI must convince the Sentinel, not just itself.

## 6.8 Earned Agency

The Sentinel gate enables a model of AI autonomy where agency is *earned through demonstrated calibration*, not granted by default or permanently restricted.

### 6.8.1 The Agency Spectrum

Current AI deployment assumes a binary: either the AI acts autonomously (with all attendant risks) or a human reviews every action (with all attendant overhead). Epistemic gating enables a middle path:

| Calibration State | Agency Level | Gate Behavior |
|---|---|---|
| Uncalibrated ($< 3$ evidence) | Conservative | Raw vectors, tight thresholds |
| Partially calibrated (3–10) | Standard | Weighted corrections applied |
| Well-calibrated ($> 10$) | Extended | Full corrections, relaxed thresholds |

Table 18: Agency levels earned through calibration evidence.

An AI system that consistently demonstrates accurate self-assessment—where grounded verification confirms that self-reports match outcomes—earns the right to more autonomous operation. An AI whose self-reports diverge from evidence has its autonomy tightened.

### 6.8.2 Auto-POSTFLIGHT

In the current workflow, POSTFLIGHT is explicitly triggered. As the system matures, automatic triggers become possible: when COMPLETION and IMPACT vectors exceed thresholds (indicating the AI judges its work complete and significant), the system can auto-trigger POSTFLIGHT without human intervention. The grounded verification track then validates whether this self-assessed completion matches objective evidence.

This creates a path toward safe autonomous operation: the AI manages its own epistemic lifecycle (PREFLIGHT $\rightarrow$ work $\rightarrow$ auto-POSTFLIGHT $\rightarrow$ POST-TEST), with the grounded track serving as an automated supervisor that detects miscalibration.

#### 6.8.3 Bootstrap-Before-CHECK

A prerequisite for earned agency is that the AI enters each transaction with adequate context. The framework enforces bootstrap-before-CHECK: before submitting to the Sentinel gate, the AI must load project context (historical findings, dead-ends, goals). This ensures gate decisions incorporate institutional memory, not just the AI's in-context knowledge.

This enforcement prevents a subtle failure mode: an AI that has been compacted (context summarized) may assess high KNOW based on its summary, but the summary may omit critical context. Bootstrap loading recovers this context before the gate evaluates readiness.

### 6.9 Summary

The Sentinel protocol implements epistemic gating:

1. **Gate, don't filter**: Prevent action when knowledge is insufficient, rather than filtering bad outputs after generation.
2. **Calibration-aware**: Apply learned corrections to compensate for systematic self-assessment biases.
3. **Investigation loop**: When knowledge is insufficient, enforce exploration until threshold is reached.
4. **Configurable**: Thresholds adjustable for different domains and risk tolerances.

The empirical result: calibrated gating reduces error rates by more than 50% compared to raw self-assessment. The learning delta (Section 5) shows that investigation produces genuine capability growth—the gate doesn't just prevent bad outputs; it enables good ones.

## 7 Discussion

### 7.1 What the Learning Delta Means

The central empirical finding—unified confidence growth across all 13 epistemic vectors during task execution—admits multiple interpretations.

#### 7.1.1 The Learning Interpretation

Our preferred interpretation: the delta measures genuine learning. AI systems begin tasks with incomplete knowledge, investigate during the noetic phase, acquire task-relevant capability, and correctly assess higher capability at task end.

Under this view, the PREFLIGHT → POSTFLIGHT delta is not a correction for miscalibration. It is the learning itself, made visible through self-assessment.

This interpretation is supported by the structure of the CASCADE workflow. AI systems that reach POSTFLIGHT have necessarily engaged with the task—reading files, exploring code, synthesizing information. This engagement produces real changes in the system's effective capability on the specific task at hand.

#### 7.1.2 The Calibration Interpretation

An alternative interpretation: capability is fixed, but self-assessment improves. The AI doesn't learn; it simply becomes more accurate about what it already knew.

Under this view, PREFLIGHT assessments are systematically conservative (perhaps due to RLHF training), and task engagement provides evidence that corrects this conservatism.

Self-assessment data alone cannot definitively distinguish these interpretations. However, the grounded verification track (Table 10) provides initial evidence: KNOW shows a gap of only −0.034 between self-assessed and evidence-derived values, suggesting POSTFLIGHT self-assessment for knowledge tracks objective outcomes accurately. The magnitude of the learning delta (+0.200 average for KNOW, +0.704 for COMPLETION), the 93.6% improvement rate across 267 clean sessions, and the 115× reduction in calibration variance (Section 5) collectively suggest more than mere assessment correction.

#### 7.1.3 The Pragmatic View

From a practical standpoint, the interpretation may not matter. Whether the AI "really learns" or "really calibrates," the intervention is the same: enforce investigation before action, measure the delta, apply corrections to future PREFLIGHT assessments.

The framework produces value regardless of which interpretation is correct.

### 7.2 Implications for AI Deployment

#### 7.2.1 Against the Execution-Layer Model

The data argues against treating AI systems as stateless execution layers. The significant learning delta suggests that task engagement fundamentally changes the AI's effective capability. Systems that skip investigation—that proceed directly from task specification to output—forfeit this capability growth.

For complex tasks, the execution-layer model may be fundamentally unsuited. AI systems may need to be designed as *learning systems* that investigate before acting.

#### 7.2.2 Epistemic Gating as Safety Mechanism

The Sentinel protocol implements a form of safety through epistemic constraint. Rather than filtering outputs for harmful content, it prevents action when knowledge is insufficient.

This addresses a class of failures that content filtering cannot catch: confident-but-wrong outputs on complex tasks. An AI that doesn't know enough to solve a problem correctly also doesn't know enough to recognize that its solution is wrong. Epistemic gating intervenes earlier—at the knowledge assessment stage—before incorrect outputs are generated.

#### 7.2.3 Calibration as Continuous Learning

The Bayesian belief system enables AI systems to improve their self-assessment over time. As delta measurements accumulate, the calibration corrections become more accurate.

This is a form of meta-learning: the system learns how to assess its own learning. Early PREFLIGHT assessments may be poorly calibrated, but the correction mechanism improves with evidence.

#### 7.2.4 The Grounding Growth Loop

The dual-track calibration system creates a reinforcing cycle between self-assessment quality and evidence accumulation:

1. The AI assesses its epistemic state (PREFLIGHT)
2. Investigation produces genuine capability growth (noetic phase)
3. Self-assessment captures the learning (POSTFLIGHT)
4. Objective evidence grounds the self-assessment (POST-TEST)

5. Calibration corrections improve future assessments
6. Better assessments lead to better-calibrated gate decisions
7. Better gate decisions lead to more appropriate investigation

Each loop iteration tightens the relationship between self-report and reality. The semantic retrieval system (Section 4) accelerates this by surfacing calibration-relevant context at PREFLIGHT: prior sessions with similar tasks and their calibration outcomes inform the baseline assessment, enabling context-sensitive corrections beyond aggregate bias.

This loop operates at a different level than RLHF's reward signal. RLHF optimizes for evaluator preferences; the grounding loop optimizes for outcome accuracy. The two are complementary: RLHF prevents harmful overconfidence, while the grounding loop prevents the overcorrection that RLHF's conservatism incentive can produce.

### 7.3 Why Would the AI Be Honest?

A natural objection: if self-assessment determines whether the AI must investigate (costly) or can proceed (efficient), why wouldn't the AI inflate its self-assessment to avoid investigation?

This objection assumes the wrong incentive structure. In the Empirica framework, self-assessment is not a report to an external evaluator—it is a routing signal for the AI's own behavior. Three mechanisms prevent gaming:

1. **Functional consequences**: Inflated KNOW at PREFLIGHT leads to premature action, which leads to poor outcomes, which the grounded verification track detects. The calibration correction then adjusts future gate thresholds upward, making it *harder* to proceed next time.
2. **Measurement, not prediction**: The framework measures what happened (learning delta, grounded calibration gap), not what the AI predicted would happen. Even if the AI inflates assessments, the delta and gap computations reveal the true calibration state.
3. **No optimization target**: Unlike RLHF, which provides an explicit reward signal the model can learn to exploit, the CASCADE workflow does not train the model. Vectors are instruction-following outputs, not gradient-optimized parameters. The model has no mechanism to strategically distort assessments across sessions.

The deeper point is that this framework sidesteps the honesty question entirely. We do not ask the AI to be honest; we measure what it does and compare it to what it said. The grounded track makes gaming detectable even if not preventable.

### 7.4 Implications for AI Alignment

#### 7.4.1 Training-Induced Conservatism

The systematic pattern of underestimation across capability vectors suggests that alignment training (RLHF, Constitutional AI) may induce broad conservatism beyond its intended targets.

RLHF specifically targets overconfidence and hallucination—training models to say "I don't know" when uncertain. Our data suggests this training may generalize: models become conservative not just about uncertainty, but about capability assessment generally.

This is not necessarily harmful. Conservative AI may be safer than overconfident AI. But it does suggest that alignment training has unintended effects that can be measured and, if desired, corrected.

### 7.4.2 Toward Safe Autonomous AI

The earned agency model (Section 6) suggests a practical path to AI autonomy that does not require solving alignment in general. Instead of asking "is this AI aligned?" (a question that may be unanswerable), we ask "has this AI demonstrated accurate self-assessment in this domain?"

The grounded verification track provides the measurement mechanism. An AI whose self-reports consistently match objective outcomes in a given domain has earned a degree of trust *for that domain*. This trust is:

- **Domain-specific**: Well-calibrated for code review does not imply well-calibrated for legal analysis.
- **Revocable**: Increasing divergence between tracks triggers tighter gating.
- **Evidence-based**: Trust is proportional to grounded verification evidence count, not to training data size or benchmark performance.
- **Continuous**: Not binary (trusted/untrusted) but graduated along the calibration spectrum.

This reframes the AI safety question from "how do we make AI safe?" to "how do we measure whether AI is operating within its epistemic bounds?" The latter question is empirically tractable.

### 7.4.3 The Engagement Dimension

The ENGAGEMENT vector—capturing collaborative quality—emerges as potentially important for alignment. Our data shows engagement is already well-calibrated (lowest learning delta at +0.022), suggesting AI systems accurately perceive collaboration quality from the start.

This raises an intriguing possibility: engagement could serve as an alignment signal. High engagement correlates with productive human-AI collaboration; low engagement may indicate misalignment, confusion, or adversarial interaction. Further research could explore whether engagement patterns distinguish aligned from misaligned interactions.

## 7.5 Toward a Cognitive Operating System

The subsystems described in this paper—epistemic gating, calibration management, context storage, agent scheduling, capability-based access control—are not independent tools. They form an integrated system with a recognizable architectural pattern: an **operating system for AI cognition**.

The mapping to POSIX/Linux is structural, not metaphorical:

Two subsystems merit detailed examination.

**Memory Management** The memory manager implements a four-layer hierarchy analogous to register/cache/RAM/disk: HOT (in-context, nanosecond access), WARM (SQLite, microsecond queries), SEARCH (semantic vector retrieval via Qdrant, millisecond), and COLD (git notes and YAML, versioned and portable). Context pressure—the epistemic equivalent of memory pressure—triggers compaction: the HOT layer is compressed into a handoff report, epistemic state persists to WARM storage, and a continuation session resumes with reduced but recoverable context. A decay mechanism provides garbage collection: long-lived lessons lose weight when new findings contradict them, scoped by domain to prevent cross-domain interference.

| OS Subsystem | Empirica Equivalent |
|---|---|
| Kernel | Sentinel + CASCADE engine |
| System call interface | MCP tools (the API surface) |
| Security module (LSM) | Sentinel gate (capability-based ACL) |
| Scheduler | Goal/subtask manager (priority, dependencies) |
| Memory manager | Context budget (HOT/WARM/SEARCH/COLD layers) |
| Interrupt handler | Hook system (PreToolUse, PostToolUse, etc.) |
| Process model | Agent sessions (isolated context, lifecycle) |
| IPC | Agent-to-agent protocol, shared findings |
| Filesystem | Storage layers (SQLite, Qdrant, git notes, YAML) |
| Boot sequence | SessionStart → bootstrap → PREFLIGHT |
| sysctl | Human-tunable thresholds (know_min, uncertainty_max) |
| /proc, sysfs | System dashboard (cognitive subsystem telemetry) |

Table 19: Structural mapping between POSIX and Empirica subsystems.

**System Observability** The system dashboard implements `/proc`-equivalent telemetry. Seven subsystem status modules—configuration, memory (zone utilization, page faults, pressure metrics), message bus (event counts, observer health), attention budget (domain allocation, strategy selection), integrity (gap detection, confabulation risk), gate state (phase, decision, vectors), and node identity (AI identity, session lineage)—aggregate into a single queryable surface. An operator can observe memory pressure rising, attention depleted in a domain, or integrity gaps widening, and intervene with the same specificity as a systems administrator reading `/proc/meminfo`.

**Routed Investigation, Gated Execution** The core architectural principle is separating investigation from action. Stochastic reasoning flows freely during the noetic phase—the AI explores, hypothesizes, and acquires knowledge without constraint. But modification of external state (writing code, editing files, deploying changes) requires passing through a deterministic gate calibrated by objective evidence. This preserves the AI's generative capacity while ensuring that actions are grounded in measured knowledge. The analogy to user/kernel separation is structural: untrusted computation runs in user space; system-modifying operations require a transition through a controlled interface.

This mapping describes deployed infrastructure, not a proposed architecture. The subsystems have been in production for over six months, processing 479,686 observations across 638 sessions (Table 6). The framework operates as middleware: it sits between the AI model and the task environment, requiring no modifications to the underlying model or hosting infrastructure. It integrates through standard interfaces (CLI, MCP tool-use server, event hooks) that attach to existing agent frameworks—providing the epistemic management layer that those frameworks currently lack.

This framing explains why all the subsystems must be coherent. An operating system without a memory manager runs until it crashes. An AI framework without context management runs until it hallucinates. An operating system without a security module lets any process access any resource. An AI framework without epistemic gating lets any confidence level trigger any action.

The implication for the broader AI ecosystem is that emerging agent frameworks (tool-use APIs, agent-to-agent protocols, orchestration layers) are building the equivalent of device drivers, network protocols, and hardware abstraction layers—but without a kernel. They assume the AI's cognitive resources are self-managing. Our data suggests otherwise: unmanaged AI cognition produces the overconfidence trap, calibration drift, and epistemic dark matter (unmeasured work).

A cognitive operating system does not make AI "conscious" or "self-aware" any more than Linux makes a CPU conscious. It manages computational resources—in this case, epistemic resources—so that the system operates within its knowledge bounds. The contribution is infrastructure, not philosophy.

## 7.6 Limitations

### 7.6.1 Functional Measurement, Not Introspection

A clarification on what we measure: the epistemic vectors are **functional assessments**—responses to structured prompts like "rate your knowledge of this task 0-1"—not phenomenological introspection. This is the same process as when an AI is asked to verify facts, estimate confidence, or perform any quantified evaluation. The AI follows instructions to produce a numeric output; no privileged access to internal states is required or claimed.

This distinction matters because critics may object that AI cannot "truly know what it knows." We sidestep this entirely: a thermometer doesn't experience temperature, yet produces useful measurements. Similarly, an AI producing consistent, calibratable vector responses is performing functional measurement regardless of whether it has introspective access.

The limitation is not that assessments are "self-reported" in the phenomenological sense, but rather that they are **instruction-following outputs** whose correlation with actual capability requires empirical validation. The Bayesian framework provides this: systematic patterns across 479,686 observations demonstrate that these instruction-following outputs track something real about epistemic state. Whether this constitutes "self-knowledge" in any deeper sense is orthogonal to the framework's utility.

### 7.6.2 Model Concentration

55% of our evidence derives from Claude (Anthropic). While we observe consistent directional effects across GPT, Gemini, and Qwen, the magnitude of effects may be model-specific.

The calibration corrections we report should be treated as Claude-specific pending cross-model replication.

### 7.6.3 Task Distribution

Our data comes from software engineering tasks—code review, implementation, debugging. The generality of findings to other domains (creative writing, analysis, conversation) is untested.

### 7.6.4 Temporal Confounding

Data collection spanned active framework development. Calibration patterns may partially reflect framework changes rather than stable AI properties.

## 7.7 Future Work

The epistemic vector framework opens numerous research directions across validation, architecture, and application domains.

### 7.7.1 External Validation (In Progress)

External validation—measuring task success against epistemic vectors without relying on self-report—is now partially underway through the grounded verification track (Table 10). Preliminary results from 7 verifications show that POST-TEST evidence collection can ground self-assessment in objective outcomes, though coverage remains limited to 27% of vectors per session.

The grounded track represents a first step, but deeper validation remains critical:

- Human evaluation of output quality correlated with epistemic assessments
- Automated test suites measuring code correctness against KNOW vectors
- Downstream task performance metrics linked to gate decisions
- A/B testing of gated vs ungated workflows on equivalent tasks
- Domain-specific evidence collectors (e.g., legal citation accuracy, medical diagnostic concordance) that provide richer grounding signals

#### 7.7.2 Mechanistic Interpretability

The hypothesized mapping between epistemic vectors and attention mechanisms (Table 4) remains untested. Collaboration with interpretability researchers could:

- Identify attention patterns that correlate with high/low KNOW assessments
- Ground UNCERTAINTY in measurable entropy or calibration properties
- Discover whether vectors emerge from specific circuit families
- Enable direct measurement rather than prompted self-assessment

#### 7.7.3 Adaptive Threshold Learning

The current threshold configuration (Table 14) relies on human judgment. Future work could:

- Learn optimal thresholds from outcome data per domain
- Implement human-in-the-loop threshold adjustment with feedback
- Develop adaptive thresholds that tighten/loosen based on error patterns
- Study threshold sensitivity—how much do outcomes vary with threshold changes?

#### 7.7.4 Learning Capability Benchmarks

The learning delta suggests a novel benchmark paradigm: measuring not what AI knows, but how well it learns. A "learning capability benchmark" would assess:

- Epistemic velocity—rate of capability growth through investigation
- Learning efficiency—knowledge gained per token of investigation
- Calibration speed—how quickly self-assessment converges to accuracy
- Transfer learning—whether epistemic gains in one domain transfer to others

#### 7.7.5 Cross-Model Studies

Systematic comparison across model families could reveal whether calibration patterns are universal or training-specific:

- Do all RLHF-trained models show similar conservatism patterns?
- How do base models vs instruction-tuned models differ in calibration?
- Can calibration learned on one model transfer to another?
- Do different architectures (dense vs MoE) show different epistemic signatures?

#### 7.7.6 Multi-Agent Epistemic Coordination (Implemented, Open Research)

The framework includes multi-agent coordination primitives: goal discovery via git notes (agents can find and resume goals created by other agents), inter-agent messaging, and a parallel orchestrator that spawns domain-specific agents (security, architecture, performance, UX). Goal lineage tracking records which agents touched a goal and when.

However, these primitives expose unresolved research questions about epistemic coordination:

- How should epistemic state propagate between collaborating agents? Current goal handoff transfers context but not calibration state.
- Can one agent's high KNOW compensate for another's low KNOW? The current model treats each agent's epistemic state independently.
- What trust mechanisms allow agents to rely on each other's assessments? Goal lineage provides attribution but not trust scoring.
- How do epistemic conflicts between agents get resolved? When two agents produce contradictory findings, no automated resolution exists.

#### 7.7.7 Long-Term Epistemic Memory (Implemented, Open Research)

CASCADE cycles are no longer session-scoped. The framework implements a four-layer memory architecture: HOT (in-context, nanosecond access), WARM (SQLite, microsecond queries), SEARCH (Qdrant semantic retrieval, millisecond), and COLD (git notes and YAML, versioned and portable). Cross-session learning persists through epistemic handoff reports that compress session state for continuation. A cognitive immune system decays lessons when new findings contradict them, with domain-scoping to prevent autoimmune attacks. Memory-informed PREFLIGHT retrieves relevant patterns (lessons, dead-ends, calibration warnings) from Qdrant at task start.

Open research questions remain:

- Optimal decay rates—current decay is keyword-triggered with fixed decay amount (0.05 per contradiction). Reinforcement-based decay modeling could improve accuracy.
- Memory transfer across domains—whether epistemic gains in one domain (e.g., security patterns) transfer meaningfully to adjacent domains (e.g., architecture review).
- Compaction fidelity—handoff reports achieve $\sim$90% token reduction but some context is inevitably lost. Measuring the epistemic cost of compaction remains informal.
- Memory capacity scaling—as projects accumulate thousands of findings and lessons, retrieval quality and storage efficiency need systematic evaluation.

#### 7.7.8 Domain Expansion: Legal as Grounded Verification Case Study

Our data derives from software engineering tasks. The grounded verification framework is domain-independent by design—the POST-TEST phase maps evidence sources to vectors, and different domains provide different evidence types. Legal work offers a particularly compelling case study because it provides naturally verifiable claims.

**Legal Evidence Collectors** Legal AI work produces claims that map directly to epistemic vectors through objective verification:

| Legal Evidence | Quality | Vectors Grounded |
|---|---|---|
| Precedent citation accuracy | Objective | know, signal |
| Statutory currency (up-to-date?) | Objective | context, state |
| Jurisdictional correctness | Objective | context, clarity |
| Applied law reasoning | Semi-objective | coherence, know |
| Case outcome prediction | Objective | impact, completion |
| Opposing counsel challenges | Objective | uncertainty, signal |

Table 20: Legal evidence collectors and the vectors they ground.

**Why Legal Grounding Is Stronger** In software engineering, grounding relies on test results (pass/fail) and git metrics (lines changed)—useful but coarse signals. Legal work provides richer grounding because:

1. **Citations are binary verifiable**: A case citation either exists and says what the AI claims, or it doesn't. This provides KNOW grounding at quality 1.0 (objective) for every citation.
2. **Statutes are version-controlled**: Legislative databases track amendments with timestamps. Whether a statute is current as of a given date is deterministic—grounding CONTEXT and STATE with no ambiguity.
3. **Cost per verification is lower**: Checking a citation against a legal database is cheaper per claim than running a test suite against a codebase. More claims verified means higher grounded coverage per session.

This suggests that the grounded verification track, currently at 27% coverage in software engineering, could achieve substantially higher coverage in legal domains—potentially grounding 8+ of the 13 vectors with objective evidence.

**Other Domains** Beyond legal, other domains offer varying levels of grounding potential:

- Creative tasks (writing, design)—low grounding potential, quality is subjective
- Conversational agents—moderate, engagement assessable via user behavior metrics
- Scientific reasoning—high, claims verifiable against literature and data
- Embodied AI—high, physical actions produce measurable outcomes

#### 7.7.9 Verification Schema Evolution: From Stochastic to Deterministic

A fundamental distinction underlies the grounded verification architecture: the AI's work and self-assessment are *stochastic inputs*—probabilistic, shaped by the model's epistemic state—while the verification logic is *deterministic output*—defined rules that produce pass/fail signals independent of the AI.

The path to reducing human oversight is systematically expanding the deterministic verification surface. We identify three tiers of verification quality:

Tier 1 checks are already implemented in the POST-TEST phase. Tier 3 is inherently bounded by the AI's own epistemic state—blindspots in the work become blindspots in verification. The critical lever is Tier 2: **domain-specific verification schemas** that define what must be checked, independent of the AI's judgment.

| Tier | What decides what to check | Example | Trust |
|---|---|---|---|
| 1. Fully automated | System (predefined) | Artifact ratios, pytest, git | Highest |
| 2. Schema-driven | Domain template | "Check all citations" | High |
| 3. AI-selected | AI judgment | "I should verify X" | Medium |

Table 21: Verification tiers by who determines check coverage.

**Schema Evolution Through Trajectory Analysis** Verification schemas need not be static. As epistemic trajectories accumulate, patterns emerge:

1. Blindspot detection (via trajectory analysis) identifies recurring gaps in AI output
2. Each detected blindspot becomes a candidate for a new deterministic check
3. The check is added to the domain verification schema
4. Future sessions apply the check automatically (Tier 2, not Tier 3)
5. The stochastic surface area—where human verification is still needed—shrinks

This creates a measurable quantity: the **epistemic entropy** of a domain's verification state, defined as the gap between what the verification schema covers and what the full output space requires. As schemas expand through trajectory iteration, entropy decreases and the need for human verification diminishes proportionally.

**Open Questions** Several questions remain for this research direction:

- **Entropy formalization**: How to formally measure the entropy state of an epistemic trajectory—as a ratio, a distribution, or requiring domain-specific formalization?
- **Convergence rate**: How many trajectory iterations are needed to achieve target coverage in a given domain? Is there a diminishing returns curve?
- **Cold-start problem**: How to bootstrap verification schemas for new domains with no trajectory history. Can cross-domain transfer help (e.g., legal citation patterns informing medical literature verification)?
- **Schema completeness**: Can we prove that a verification schema is "complete" for a given domain, or is the output space inherently open-ended?

This direction connects the earned agency model (Section 6) to a concrete mechanism: autonomy increases not because we trust the AI's self-report, but because the deterministic verification layer expands to cover more of the output space. Trust follows coverage, not confidence.

#### 7.7.10 Alignment Applications

The ENGAGEMENT vector's near-zero delta suggests potential alignment applications:

- Can engagement patterns distinguish aligned from misaligned interactions?
- Does low engagement correlate with adversarial or confused users?
- Can epistemic gating prevent harmful outputs by detecting low-engagement contexts?
- How do jailbreak attempts manifest in epistemic vectors?

**DENSITY as Safety Signal** Preliminary observations suggest extreme DENSITY values (approaching 1.0) correlate with AI system freezes or halts. This could indicate:

- **Cognitive overload**: Information packing exceeds processing capacity
- **Natural circuit breaker**: A safety mechanism preventing action under information saturation
- **Adversarial signature**: Prompt injection attacks often maximize information density to overwhelm context windows

The relationship between information density and system stability merits investigation. If high DENSITY reliably predicts system failure or adversarial input, it could serve as an early-warning signal for epistemic gating.

#### 7.7.11 Real-Time and Streaming Applications

Production deployment may require lower-latency epistemic assessment:

- Streaming epistemic updates during generation
- Latency-aware gating that balances thoroughness with responsiveness
- Incremental calibration without full CASCADE cycles
- Edge deployment with limited computational resources

### 7.8 Summary

The epistemic vector framework and its empirical validation suggest that AI self-assessment is meaningful, measurable, and practically useful. The learning delta demonstrates that AI capability genuinely changes during task engagement—supporting the noetic-before-praxic principle.

The limitations are real: limited grounded verification coverage, model concentration, domain specificity. The dual-track calibration system partially addresses the self-referential measurement concern, but full external validation remains incomplete. The framework provides a foundation for systematic investigation of AI epistemic state. We release all data for replication and extension.

## 8 Conclusion

### 8.1 Summary of Contributions

This paper presents Empirica, a framework for AI epistemic self-assessment, and validates it with substantial empirical evidence.

**Conceptual contribution:** We articulate the *noetic-before-praxic* principle—AI systems must investigate before acting—and argue that treating AI as an execution layer is a category error that produces systematic failures.

**Theoretical contribution:** We propose a 13-dimensional epistemic vector representation capturing what AI systems know, what they can do, and how ready they are to act. This constitutes a computational epistemology applicable to any information-processing system managing its own knowledge state.

**Empirical contribution:** We present 479,686 evidence observations from 638 production sessions demonstrating:

- Unified confidence growth across all 13 vectors during task execution
- Average learning delta of +0.200 (KNOW vector) with 93.6% of sessions showing improvement
- Calibration convergence: 115× variance reduction as evidence accumulates
- Systematic patterns supporting functional validity of self-assessment

**Practical contribution:** We release Empirica as open-source infrastructure for epistemic gating, dual-track calibration, and the 4-phase CASCADE workflow (PREFLIGHT → CHECK → POSTFLIGHT → POST-TEST).

### 8.2 The Core Claim

The central claim is simple: **AI systems that investigate before acting outperform AI systems that act without investigating**.

The evidence is in the delta. AI capability at task-end exceeds capability at task-start, across all measured dimensions, because investigation produces learning. The overconfidence trap—acting on insufficient knowledge—is not just a failure mode to avoid; it is a learning opportunity foregone.

### 8.3 Implications

**For AI practitioners:** The execution-layer model is insufficient for complex tasks. Design AI systems as learning systems with epistemic gates that enforce investigation.

**For AI researchers:** Self-assessment is measurable and meaningful. The systematic patterns we observe invite replication and extension.

**For AI safety:** Epistemic gating offers a complementary approach to content filtering—preventing action when knowledge is insufficient rather than filtering harmful outputs after generation. The earned agency model provides a path to safe autonomous AI where trust is proportional to demonstrated calibration accuracy, domain-specific, and revocable.

### 8.4 The Delta

We close with the finding that motivated this work:

| Phase | Mean Assessment |
|---|---|
| PREFLIGHT | 0.663 |
| POSTFLIGHT | 0.863 |
| **Delta** | **+0.200** |
| **Improvement Rate** | **93.6%** |

Table 22: The learning delta: capability growth through investigation.

This is the learning that happens when AI systems are allowed—and required—to investigate before acting. It is the capability growth that execution-layer deployments forfeit. It is the empirical signature of noetic-before-praxic.

The delta is real. The learning is real. The framework calibrates—variance drops 115× as evidence accumulates. The framework works.

**Data Availability**: Full dataset (479,686 evidence observations across 638 sessions) available at Zenodo: https://doi.org/10.5281/zenodo.18237263. Empirica framework and analysis code: https://github.com/Nubaeon/empirica.

**Acknowledgments**: This paper was developed in collaboration with Claude (Anthropic), which contributed to drafting, analysis framing, and iterative refinement across the research process.

## References

Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, Andy Jones, Anna Chen, Anna Goldie, Azalia Mirhoseini, Cameron McKinnon, et al. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

Gagan Bansal, Tongshuang Wu, Joyce Zhou, Raymond Fok, Besmira Nushi, Ece Kamar, Marco Tulio Ribeiro, and Daniel Weld. Does the whole exceed its parts? The effect of AI explanations on complementary team performance. In *Proceedings of the 2021 CHI Conference on Human Factors in Computing Systems*, pages 1–16, 2021.

Tristan Bricken and Anthropic. On the biology of a large language model: Emergent introspective awareness. *Transformer Circuits Thread*, 2025. https://transformer-circuits.pub/2025/introspection/index.html.

Stephen M Fleming and Raymond J Dolan. The neural basis of metacognitive ability. *Philosophical Transactions of the Royal Society B: Biological Sciences*, 367(1594):1338–1349, 2012.

Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *International Conference on Machine Learning*, pages 1050–1059. PMLR, 2016.

Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International Conference on Machine Learning*, pages 1321–1330. PMLR, 2017.

Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tran-Johnson, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *arXiv preprint arXiv:2302.09664*, 2023.

Vivian Lai, Chacha Chen, Q Vera Liao, Alison Smith-Renner, and Chenhao Tan. Towards a science of human-AI decision making: A survey of empirical studies. *arXiv preprint arXiv:2112.11471*, 2021.

Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in Neural Information Processing Systems*, 35:27730–27744, 2022.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.

Mark Steyvers and Garrett Peters. Metacognition in large language models: Current landscape and open questions. *Current Directions in Psychological Science*, 2025. doi: 10.1177/09637214251391158.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in Neural Information Processing Systems*, 35:24824–24837, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *Advances in Neural Information Processing Systems*, 36, 2023a.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2023b.